Intelligent Service Robotics (2023) 16:195–212
https://doi.org/10.1007/s11370-023-00458-6

ORIGINAL RESEARCH PAPER

HTM-EC

Hierarchical Topology Map with Explicit Corridor for global path planning of mobile robots

Jeong-woo Han[1] · Soo Jeon[1] · Hyock Ju Kwon[1]

Received: 25 September 2022 / Accepted: 23 January 2023 / Published online: 16 February 2023

Abstract

One way to construct a global path for a mobile robot is to extract the equidistant points from obstacles in an occupancy grid map (OGM) as its backbone of thin lines (skeleton) and find an appropriate path from them. Such skeleton-based methods are advantageous as they provide complete path planning, i.e., a feasible path is guaranteed to be found if it exists and often allow safe distances from obstacles with light computation. However, a skeleton path needs to be properly refined to avoid excessive bypasses, which inherently exist, and get an optimal path. This paper introduces hierarchical topology map with explicit corridor (HTM-EC) as a new skeleton-based strategy for global path planning with path optimization. HTM-EC is a skeleton-based hierarchical graph structure with a topology graph and the corresponding skeleton of a map, which can be an alternative map expression of an OGM. A skeleton path can be quickly obtained from the topology graph of HTM-EC as needed with the light computation facilitated by the significantly reduced size of searching space. The novelty of the paper lies on, first, the formulation of HTM-EC structure, and second, the refinement method of the given skeleton path with cost optimization by using corridor discretization and dynamic programming. For cost optimization, we incorporate the allowable speed that limits the navigation speed in narrow areas for safe operation, which enables a time-optimal path with the cost of traveling time. Simulations and experiments were conducted with real OGMs and a mobile robot to validate the proposed global planning method.

Keywords Global path planning · Autonomous mobile robot · Topological maps · Skeleton · Voronoi graph · Explicit Corridor

1 Introduction

Global path planning is one of the core problems for mobile robot navigation, which is to find a safe and efficient path in known environments. Considering an occupancy grid map (OGM) is the most widely used form of the environments [32], some conventional grid-based approaches that directly use an OGM for global path planning, such as artificial potential field (APF) [16], Dijkstra [6], A* [11], or navigation functions [4], have been successfully implemented and become popular. One reason for popularity of such methods can be due to customizable costs and optimality based on the potential of grids, enabling useful criteria of navigation: repulsive and attractive force in APF, shortest distance in Dijkstra and A*, minimized time in fast marching [9,29]. The potential of grids can be easily incorporated together [23,24,30], creating other potentials.

However, when a map becomes large and complex, the computation time of such grid-based approaches becomes critical due to the rapidly increasing size of the environment (e.g., quadratic for 2-d OGM). Thus, many other approaches for global path planning have also been extensively studied to mitigate the computational cost by converting OGMs into graphs to reduce the searching space for a global path. Probabilistic road maps (PRMs) [13,15], for example, create a sparse graph by spreading out random sample points in the free space of a map and connecting them. Rapidly exploring random trees (RRTs) [18,19]) build a space-filling tree graph in a space. Both methods efficiently reduce searching spaces leading to light computation. Safety in those sampling-based graphs is often treated by simply inflating the obstacles in a map. Moreover, their main drawback is incompleteness, which means they do not guarantee to find a feasible path.

✉ Jeong-woo Han
jeong-woo.han@uwaterloo.ca

[1] University of Waterloo, Waterloo, ON, Canada

Such issues can be even worse in a maze-like environment where many narrow and complex corridors exist.

Among the methods that use sparse graphs of an OGM, skeleton-based approaches [2,8,31,33] can be advantageous for complete path planning [5,25], which means that it is guaranteed to find a feasible path if it exists, as the skeleton is the abstract expression of the free space of a map. Another benefit of those approaches is safety because the skeleton of a map is a set of equidistant points from obstacles (e.g., corridor walls) in a map providing the largest clearance of the resulting path (thus safest) from the obstacles. However, the main drawback of such skeleton-based approaches is that the resulting path is often far from optimal [2]. This is particularly true when a path is obtained merely from the skeleton, as the skeleton path often has unnecessary turns around intersections of corridors and provides too far waypoints from obstacles in open spaces that can be too conservative. Thus, the refinement of a skeleton path is essential to avoid unnecessary bypasses and sharp turns and to get the optimal path while ensuring safety. This is particularly true when a path is obtained merely from the skeleton. Thus, the refinement of a skeleton path is essential to avoid unnecessary bypasses and sharp turns and to get the optimal path while ensuring safety.

One way to refine the skeleton path into an optimal path is to reduce the number of edges of the skeleton path [3]. They iteratively refined the skeleton path by the corner-cutting technique that incorporates adjacent two edges into a single edge if the single edge can keep the (customizable) minimum clearance from obstacles. The iterative removal of the edges results in the path with the minimum number of edges that becomes the shortest path with the minimum clearance. However, their method requires the use of the original map to check the minimum clearance throughout the corner-cutting technique, which may degrade their computation efficiency with large maps.

Explicit corridor [8,33] is another skeleton-based method that exploits the skeleton of a map and its clearances to create a corridor map along with the skeleton. For path planning, they first obtained a path from the skeleton and then created the corridor map to reduce the searching area. In the corridor map, they created another graph by sectionalizing the corridor map with triangulation that has its vertices on the boundary. Then, the shortest path can be obtained by connecting some vertices on the boundary from the start to the goal. The corridor map can be shrunk with the minimum clearance for safety. This method may be beneficial for large maps because it does not use the original map once the corridor map is created. However, it has not been implemented with a real robot as the method was intended for the virtual world (i.e., polygon world).

More recent works [7,34] combine the skeleton-based approaches and sampling-based approaches in such a way that the skeleton narrows down the sampling space. They planned a skeleton path first, then refined the path using probabilistic approaches such as RRT. The main benefit of those methods is the computation efficiency because of the reduced size of samples near the skeleton. Yet, they did not generate an optimal path showing excessive distances from the wall [7] or lacked concerns of safety which is one of the main benefits of the skeleton-based approaches.

While a skeleton path is refined with different approaches as mentioned above, the minimum clearance is commonly used for a safe path in many works [2,8]. With ensuring the minimum clearance for the means of safety, the shortest path criterion is often used for an efficient (or optimal) path with the hope that it would provide the least time to reach the goal. However, the minimum clearance needs to be large when operating a mobile robot at high speeds because there are increased chances for collision with corridor walls. Yet, enlarging the minimum clearance for safer operation may cause other issues, such as the elimination of narrow areas of a map that were once accessible with the smaller minimum clearance if the robot moves slow. This implies that there should be some compromise of the operation speed with clearances to obstacles.

In this paper, we propose a complete global path planning method with a skeleton-based graph structure using the hierarchical topology map (HTM) [10]. HTM is a hierarchical map expression that consists of a topology graph and metric points of a skeleton at different layers. In HTM, the relation between the topology graph and the skeleton is marked in the topology graph in a new tuple of edge expression. The cost of travel along the skeleton is also marked with the accumulated cost at the topology. Then, a path can be obtained from HTM first by planning on its topology graph (higher layer) based on the accumulated cost and then by tabulating the corresponding metric points on the skeleton (lower layer). This provides an on-skeleton path (i.e., a path on the skeleton), which contains inherent bypass for some parts of the path and needs further refinement.

Therefore, the problem we tackle in this paper is how we can refine a skeleton path to an optimal (off-skeleton) path by minimizing a certain cost under the HTM structure. To achieve this, we propose a formal formulation of an extended version of HTM to have an optimization step in the path planning procedures. Inspired by the concept of the explicit corridor [8], we add the boundary points of each skeleton point into HTM to define an extended HTM named as the hierarchical topology map with explicit corridor (HTM-EC). Once an on-skeleton path is obtained in HTM, a set of points is created such that the boundary points of HTM-EC discretize the corridor space along the previously obtained on-skeleton path, and the on-skeleton path is optimized into a metric, off-skeleton in the discretized space by using dynamic programming. We assume a map is given in advance, and one-time construction of HTM-EC is per-

formed as a part of initialization when a mobile robot boots up: the HTM-EC is stored in the memory, and path planning is performed only on HTM-EC instead of conventional maps such as an occupancy grid.

For the optimization cost, we adopt the allowable speed in the problem of global path planning for time-minimization as the objective. The allowable speed limits navigation speed depending on the clearance to obstacles for safety. Unlike other global path planning methods that use the shortest distance with the minimum clearance as the means of objective and safety, respectively, the allowable speed enables us to get a time-minimized path as the optimal path, while safety is incorporated in it.

We summarize the main contributions of our work as follows:

1) introduction of HTM-EC and the associated minimum-cost path planning with reduced size of searching space for fast computation;
2) refinement method of a skeleton-path based on cost optimization with the dynamic programming;
3) robot implementation of the allowable speed of navigation into the cost function for a time-minimizing global path;

The effectiveness of our HTM-EC as a new efficient and safe path planning method has been verified by simulation and experimental results using several real maps.

The rest of the paper is organized as follows: In Sect. 2, we introduce the hierarchical topology map as the related work. Some technical details are addressed to enhance the understanding of the readers as this paper is an extended work. In Sect. 3, we explain our proposed method that contains an extended version of HTM and incorporates allowable speed of navigation. Section 4 shows simulation results with different realistic maps primarily focusing on optimality and computation efficiency. Section 5 presents the implementation on a real robot, including ROS simulation and experiments to verify the time-minimized path allowable speed of navigation. We summarize our work in Sect. 6.

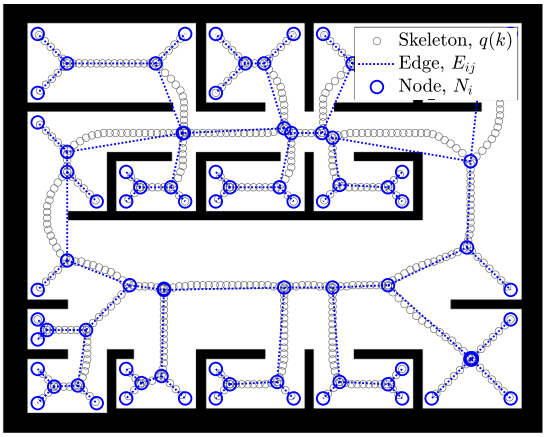
2 Related work: Hierarchical Topology Map (HTM)

The hierarchical topology map (HTM) ([10]) is a map expression that consists of two layers in a hierarchy: the skeleton of a map (lower layer) and the topology graph of the skeleton (higher layer). Skeleton in the lower layer is a morphological expression of a map with a set of equidistant points to the nearest obstacles, which can be obtained from various methods such as Voronoi diagram [22], image thinning [36], or distance fields transform [35]. We used the combination of distance field transform and the image thinning to get the skeleton, which resulted in the consecutive points on the cells of OGM. The topology graph of HTM can be obtained from the skeleton by extracting nodes from the intersection or terminal points of the skeleton and edges from the nodes' connectivity. Figure 1a visualizes an example of the HTM over a simple map, where the consecutive empty gray circles are the skeleton points, $q(k)$, the empty blue circles are the nodes, N_i, and the blue dashes are the edges, E_{ij}.

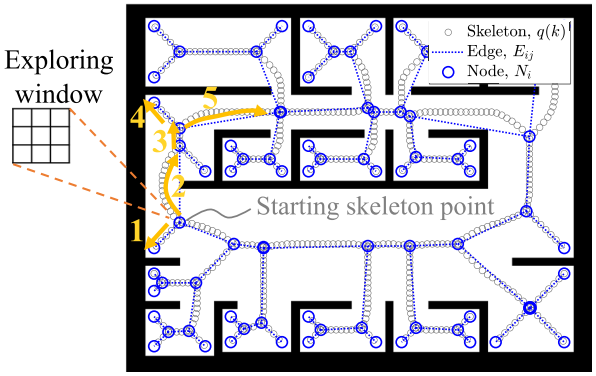
When the topology graph is constructed, the skeleton points are reordered such that adjacent points have consecutive indices. The reordering process is achieved by exploring all points in the skeleton, starting from the point with the maximum clearance, to check whether a skeleton point is terminal, intersecting with other skeleton points, or merging to a previously explored intersecting point. When a skeleton point falls into one of these three categories, it is marked as a node creating an edge. For each skeleton point being explored, its adjacent skeleton points can be found by using the window of eight neighboring cells (i.e., 3×3 window).

Let us detail such processes with an example for the exploration along with the skeleton shown in Fig. 1b. The yellow arrows in the figure visualize the orders of the exploration up to the fifth, beginning from the starting skeleton point. The starting point has three skeleton points in its neighboring cells as there are three skeleton branches at the point. These three skeleton points are added to the exploration queue, and one of them is selected for the next point to be propagated and explored. Such propagation leads to the addition of newly found skeleton points to the queue and the removal of the currently exploring point. When the terminal point is found after the first exploration, we can jump back to the starting point and continue the reordering process to the next branch (now along with the arrow labeled 2). This is because that the remaining points in the queue are the two neighboring skeleton points of the starting point. The exploration continues along with the orders as numbered by the arrows, the jumping-back situation occurs when a terminal point is found at the end of the fourth exploration (the arrow marked 4) and allows the next exploration as directed by the fifth arrow. Similarly, we can jump to the most recently added point in the queue whenever terminal or merging points are found and continue the reordering process until the queue becomes empty. Algorithm 1 in Appendix explains these processes. The particular example of Fig. 1 has 2,934 skeleton points which generated 85 nodes and 86 edges.

After reordering, we will denote the kth skeleton point by $q(k)$. If a skeleton point is either terminal or intersecting, the point is marked as a node and its connection to the previously found node is checked to form an edge. For each edge found, we keep track of the explored skeleton points in order. Thus,



(a) Example of HTM over a simple map



(b) Example of the reordering process to construct the topology graph from the skeleton

Fig. 1 Illustration of the hierarchical topology map and its construction from the skeleton of a map

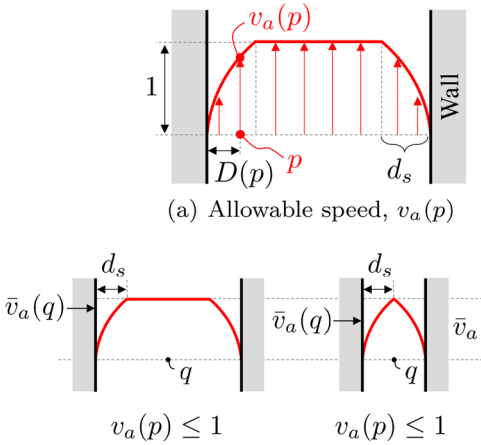
for each edge, denoted by E_{ij}, we associate four elements as

$$E_{ij} := (N_i, N_j, I_{ij}, C_{ij}) \tag{1}$$

where N_i and N_j are the nodes for the edge, I_{ij} is the set for indices containing all the skeleton points within the edge E_{ij}, and C_{ij} is the cost of the edge. The computation of C_{ij} will be explained later [by Eq. (10)].

Using the above definition of the HTM, the topology graph can be defined as an weighted undirected graph which is denoted by $G(\mathcal{N}, \mathcal{E})$ where $\mathcal{N}$ is the set of nodes and $\mathcal{E}$ is that of edges. For each E_{ij}, the information on the associated skeleton points is specified through the index set I_{ij}. By denoting $\mathcal{Q}$, the set of the ordered skeleton points q (for the rest of the paper, the term *skeleton* will be considered the *ordered skeleton*), the HTM, denoted by M_{ht}, can be defined as the combination of the topology graph G, and the skeleton, $\mathcal{Q}$ as follows.

$$M_{\text{ht}} := \{G, \mathcal{Q}\} \tag{2}$$



(a) Allowable speed, $v_a(p)$

(b) $v_a(p)$ shape with different width of corridors

Fig. 2 Allowable speed of navigation

The total number of skeleton points and that of nodes in the HTM will be denoted by n_Q and n_N, respectively.

3 Methodology

In this section, we propose a strategy of global path planning for a mobile robot based on the HTM. We first introduce an extended version of HTM with the concept of explicit corridor [8], named hierarchical topology map with explicit corridor (shortly HTM-EC). Then, we will explain how to plan a path with HTM-EC. While only an on-skeleton path is obtained with the original version of HTM, the proposed method with HTM-EC provides an off-skeleton path that is refined from the on-skeleton path.

3.1 HTM-EC: Hierarchical Topology Map with Explicit Corridor

Due to the ways in which the skeleton points are constructed, any skeleton point, $q(k) \in \mathcal{Q}$ where $k \in \{1, 2, 3, \ldots, n_Q\}$, has at least two closest points to the obstacles (e.g., walls). Those points were used in defining explicit corridor with a tuple of four elements: a skeleton point, the clearance at the skeleton point (i.e., distance to the nearest obstacle), the left closest point and the right closest point from the skeleton point. We apply explicit corridor to a skeleton point of $q(k)$, resulting in a tuple as

$$b(k) := \{q(k), r(k), O(k)\} \tag{3}$$

where $r(k)$ is the distance from the skeleton point $q(k)$ to the nearest obstacles, and $O(k)$ is the set of the closest obstacle positions. More formally, $O(k) := \{o_m(k) | m \in$

$\{1, 2, \cdots, n_o(k)\}\}$, where $n_o(k)$ is the number of the closest obstacle points from $q(k)$. The size of $O(k)$ can vary depending on the number of closest obstacles at $q(k)$. Note $q(k) \in \mathbb{R}^2$ (or the position in 2D) and $r(k) \in \mathbb{R}$.

$o_m(k) \in O(k)$ can be found when a skeleton point $q(k)$ is constructed into the HTM. Since the distance to the closest obstacles $r(k)$ is given for each $q(k)$, we can check out if the obstacle cells of the original map are matched with the distance, that is

$$o_m(k) = \{o(x, y) \mid |o(x, y) - q(k)| = r(k)\} \tag{4}$$

where $o(x, y)$ is a point on obstacles at (x, y). In practice with a grid map where each cell of obstacles has integer indices (x, y), the closest obstacle points $o_m(k)$ may not have exact $r(k)$ with a cell size. To incorporate such bias, we used a modified equation with a scaled cell size to get $o_m(k)$ in a grid map, that is,

$$o_m(k) = \{o(x, y) \mid |o(x, y) - q(k)| \in [r(k) - r_g, r(k) + r_g]\} \tag{5}$$

where r_g is the scaled size of each grid cell into the same unit as $r(k)$, e.g., in meter.

Now, let us denote $\mathcal{B}$ the set of $b(k)$ for all skeleton points, i.e., $\mathcal{B} := \{b(k) | k \in \{1, 2, 3, \cdots, n_Q\}$. Then, we can define hierarchical topology map with explicit corridor, denoted by $M_{\text{ht}}^{\text{ec}}$, as the set of G and $\mathcal{B}$

$$M_{\text{ht}}^{\text{ec}} := \{G, \mathcal{B}\}. \tag{6}$$

3.2 Formulation of cost function

In this section, we introduce the formulation of each cost $c(q(k))$. Obviously, a mobile robot needs to operate at reduced speeds when there are close obstacles in the surroundings, such as walls in narrow corridors. It is natural to consider the speed reduction into the cost function, as it can affect the time of traveling for the robot. Accordingly, we introduce the *allowable speed* of navigation as the maximum speed allowed to a mobile robot such that it can make a complete stop safely before colliding with the closest obstacle under the maximum deceleration rate $a_{\max}$. Specifically, the allowable speed of navigation $v_a(p)$ at a point p is given by the relative speed normalized by $v_{\max}$ (i.e., the maximum speed of the robot in the absence of any obstacles) as

$$v_a(p) := \begin{cases} \frac{\sqrt{2a_{\max} D(p)}}{v_{\max}}, & \text{if } D(p) < d_s \\ 1, & \text{otherwise} \end{cases} \tag{7}$$

where $D(p)$ is the distance to the closest obstacle at p, and $d_s = \frac{v_{\max}^2}{2a_{\max}}$ is the safe distance with the kinematic constraints of the robot. Note that $v_a(p)$ has its value in $[0, 1]$ where the value of 1 corresponds to $v_{\max}$. Also, note that when p is a skeleton point, (i.e., $p = q(k)$), then $D(p) = r(k)$. The shape of $v_a(p)$ across a corridor is shown in Fig. 2a, and the shape depends on the width of the corridor as shown in Fig. 2b.

When a mobile robot moves through a corridor cross section of a skeleton point $q(k)$, the actual point that the robot will pass varies across the corridor depending on the given path. Thus, it will be natural to use a representative value that can capture how easy it is to navigate passing through the corridor cross section. To quantify such ease of navigation at a skeleton point $q(k)$, we can average the *allowable speed* across its width as

$$\bar{v}_a(q(k)) := \frac{1}{D(q(k))} \int_0^{D(q(k))} v_a(p) \mathrm{d}p \tag{8}$$

where $\bar{v}_a(q(k))$ represents the mean value of the achievable maximum speeds across the cross section of the corridor at the skeleton point $q(k)$. We define this quantity as the cost value at the skeleton point $q(k)$ as

$$c(q(k)) := \frac{2\|q(k+1) - q(k)\|_2}{\bar{v}_a(q(k+1)) + \bar{v}_a(q(k))} \tag{9}$$

where the cost, $c(q(k))$, can be regarded as the averaged time expectation of traveling in the corridor enclosed by $q(k)$ and $q(k+1)$. The edge cost C_{ij} for en edge E_{ij} in Eq. (1) can be obtained by summing up all the $c(q(k))$ in the range of $k \in I_{ij}$ as follows.

$$C_{ij} := \sum_{k \in I_{ij}} c(q(k)) \tag{10}$$

3.3 Path planning using HTM-EC

Path planning with HTM-EC consists of three steps: insert start and goal into the HTM, perform topology planning from the topology graph and corresponding skeleton planning, and refine the skeleton path into an optimal metric path. These planning steps can be performed solely on HTM-EC, and the original grid map is not used. The following subsections address each of the planning steps.

3.3.1 Addition of start and goal nodes

The first step for planning a path with HTM-EC is to choose the start and the goal from the ordered skeleton set $\mathcal{Q}$ and incorporate them into the topology graph G. Assume that we are given the goal position and the start (or the current) position of the robot, which are denoted by p_G and p_R,

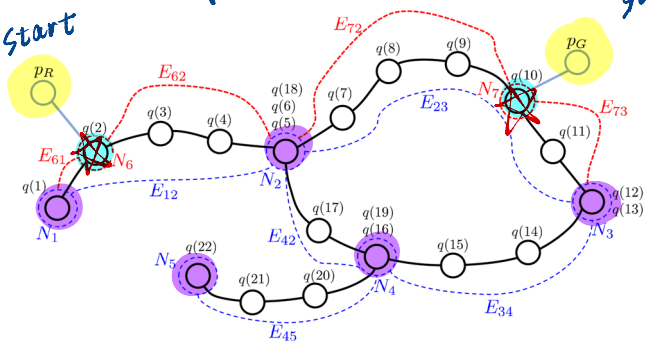


Fig. 3 Addition of start and goal on HTM for topological path planning

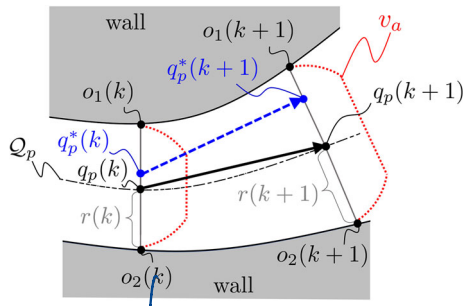


Fig. 4 Optimization of the given skeleton path in a corridor section

respectively. Then, we can first identify the closest skeleton points from p_R and p_G. Let us say these two points are $q(k_R) \in \mathcal{Q}$ and $q(k_G) \in \mathcal{Q}$, respectively, where k_R and k_G are the related indices in $\mathcal{Q}$. If $q(k_R)$ and $q(k_G)$ are not in $\mathcal{N}$, they are appended to the existing nodes as N_{n_s+1} and N_{n_s+2}. This will, in turn, generate four additional edges: two for N_{n_s+1} with its two neighboring nodes, and two for N_{n_s+2} likewise. Figure 3 visualizes these processes with a simple example of HTM that has the topology graph G with five nodes and their edges shown in blue and associated skeleton points, $q(k)$, shown in black. With the given p_R and p_G, the two additional nodes, N_6 and N_7, and their four additional edges are shown in red.

The insertion of the start and goal is dynamically performed whenever there is a planning request, while the topology graph $\mathcal{G}$ and the ordered skeleton set $\mathcal{Q}$ remain intact.

3.3.2 Planning an on-skeleton path with HTM

Let us denote the topology graph with the additional nodes and edges by $G' = \{\mathcal{N}', \mathcal{E}'\}$. Then, we can find the shortest path *within the skeleton* by minimizing the total cost of edges for the path, i.e.,

$$\min \sum_{i,j} C_{ij} \quad \forall (i, j) \text{ connecting } N_{n_s+1} \text{ and } N_{n_s+2} \tag{11}$$

which can be solved on G' by applying a conventional shortest path-finding technique (we used Dijkstra method). Let us denote the set of edges that solves Eq. (11) by $\mathcal{E}_p \in \mathcal{E}$. The corresponding skeleton path, say $\mathcal{Q}_p$, can be obtained by stacking up all the skeleton points associated with $\mathcal{E}_p$ using the indices I_{ij} stored in each edge of $\mathcal{E}_p$.

3.3.3 Optimization of the given skeleton path into off-skeleton path

As the resulting path $\mathcal{Q}_p$ is an on-skeleton path, it still needs to be refined. Specifically, the path resulting from $\mathcal{Q}_p$ may have sharp corners near the intersections of corridors and have unnecessary clearance from the walls. Thus, we need to refine $\mathcal{Q}_p$ to get an off-skeleton path by another optimization routine. The basic idea is to find the optimal amount of lateral deviation from each skeleton point. In other words, at each skeleton point $q(k)$, we can obtain the cost at any point in its traverse space to the nearest obstacles. Then the optimal path would be the one that goes from the given start and goal points and minimizes the path integral of the cost along the given skeleton path $\mathcal{Q}_p$.

Letting n_p be the total number of skeleton points of $\mathcal{Q}_p$, a skeleton point, denoted by $q_p(k) \in \mathcal{Q}_p$ where $k \in \{1, 2, \cdots, n_p\}$, immediately provides us with the corresponding tuple $b(k)$ given by Eq. (3). Recalling $o_m(k) \in O(k)$ in Eq. (3) refers to the position (x, y) of the mth closest point from the skeleton point $q_p(k)$, we can draw a line, $\overline{q_p(k)o_m(k)}$, for each $o_m(k)$. See Fig. 4 for the illustration. In the figure, two skeleton points, $q_p(k)$ and $q_p(k+1)$, are given from the skeleton path $\mathcal{Q}_p$, and the lines to their closest points $o_1(k)$, $o_2(k)$, $o_1(k+1)$ and $o_2(k+1)$ are drawn. Please note that $\overline{q_p(k)o_1(k)}$ and $\overline{q_p(k)o_2(k)}$ from a skeleton point are not necessarily colinear although they look so in Fig. 4.

One can notice in Fig. 4 that we drew the profile of the allowable speed, v_a, on each line of $\overline{q_p(k)o_1(k)}$ and $\overline{q_p(k)o_2(k)}$. If corridors are sufficiently wide, a robot may not need to stick to the skeleton path $\mathcal{Q}_p$ which consists of the farthest points from the obstacles. Rather, the robot can go closer to the wall to get its goal faster with some compromise between shorter paths and reduced speed, both of which can result from smaller clearances from the wall. This suggests us that we can improve the skeleton path $\mathcal{Q}_p$ by considering point, say $q_p^*(k)$ lying on lateral branch lines of $q(k)$ (i.e., $\overline{q_p(k)o_1(k)}, \overline{q_p(k)o_2(k)}, \ldots$). See Fig. 4 for the example of $q_p^*(k)$.

By choosing $q_p^*(k)$ properly for all $k \in \{1, 2, 3, \cdots, n_p\}$, we can obtain the optimal path that is off-skeleton, denoted by $\mathcal{Q}_p^*$, along the corridor of the on-skeleton path $\mathcal{Q}_p$. The procedures of determining $q_p^*(k)$ can be explained in a more detail by an example shown in Fig. 5. In

Fig. 5a, a corridor is drawn along by the given skeleton path $\mathcal{Q}_p$ (the red dashed line) with arbitrary widths. For each skeleton point $q_p(k)$, we drew the lateral branch lines $\overline{q_p(k)o_1(k)}$, $\overline{q_p(k)o_2(k)}$, $\overline{q_p(k+1)o_1(k+1)}$ and $\overline{q_p(k+1)o_2(k+1)}$. Let n_s be the number of segments of the line $\overline{q_p(k)o_m(k)}$ for each $m \in \{1, 2, \ldots\}$ and h be the order of the discretized point counted from $q_p(k)$ (e.g., $n_s = 2$ in Fig. 5). If these segments are equally segmented (which we assume in this paper), the hth point on the line $\overline{q_p(k)o_m(k)}$, denoted by $s_m(k, h)$, can be computed by Eq. (12). The total set of all the discretized points, denoted by $S(k)$, can be defined as Eq. (13).

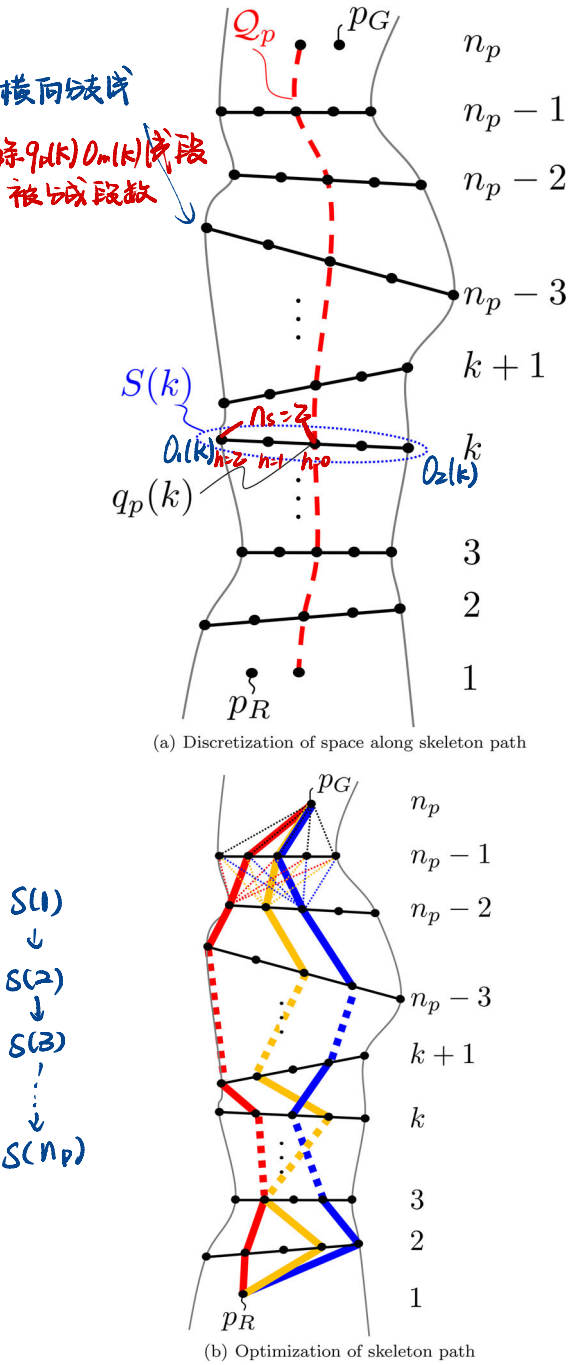
$$s_m(k,h) = \frac{h}{n_s}(o_m(k) - q_p(k)) + q_p(k) \tag{12}$$

$$\begin{aligned} S(k) &:= \{q_p(k), s_1(k,1), \cdots, s_1(k,n_s), \cdots, s_m(k,1), \cdots, \\ &\quad s_m(k,n_s), \cdots, s_{n_o(k)}(k,1), \cdots, s_{n_o(k)}(k,n_s)\} \\ &:= \{s(k,1), s(k,2), \cdots, s(k,g), \cdots, \\ &\quad s(k, n_o(k)n_s + 1)\} \end{aligned} \tag{13}$$

$n_o(k)$ is the number of closest obstacles as defined earlier (e.g., $n_o(k) = 2$ for all k in Fig. 5a). Please notice that in Eq. (13), we reformed $s_m(k,h)$ to $s(k,g)$, where $g \in \{1, 2, 3, \cdots, n_o(k)n_s + 1\}$, such that $s(k,1) = q_p(k)$ followed by $s_m(k,h)$'s for all h and m. We can obtain $S(k)$ for all skeleton points $q_p(k)$, which can be regarded as the candidates for $q_p^*(k)$. For the start and goal points, p_R and p_G, we can set $S(1) = \{s(1,1)\} = \{p_R\}$ and $S(n_p) = \{s(n_p,1)\} = \{p_G\}$ since the two points must be on the final path.

Since the corridor is discretized as the set of $S(k)$ for all $k \in \{1, 2, \ldots, n_p\}$, we can consider $s(k,g) \in S(k)$ and $s(k+1, g') \in S(k+1)$ for some g and g' as the candidates of the optimal points, $q_p^*(k)$ and $q_p^*(k+1)$, respectively. Then, the paths from the start to the goal can be represented as the combinations of the consecutive pairs of $s(k,g)$ and $s(k+1,g')$ as shown with different colors in Fig. 5b with the ranges of $k \in \{1, 2, \cdots, n_p - 1\}$, $g \in \{1, 2, 3, \cdots, n_o(k)n_s + 1\}$ and $g' \in \{1, 2, 3, \cdots, n_o(k+1)n_s + 1\}$. Then, we can choose a path that minimizes the total cost from the start to the goal as the optimal path. Such a path can be found by a straightforward application of the dynamic programming. More specifically, the procedure goes as follows.

Let U_k^g be the minimum cumulative cost to the goal from $s(k,g)$ and $u_k^{gg'}$ be the cost from $s(k,g)$ to $s(k+1,g')$. According to the Bellman's principle of optimality [1], the minimum cumulative cost U_k^g can be said as the minimum sum of the cost from the current (i.e.,$s(k,g)$) to the next point (i.e., $s(k+1,g')$) together with the minimum cumulative cost from the next point (i.e., $s(k+1,g')$) to the goal, which can be written as follows.



(a) Discretization of space along skeleton path

(b) Optimization of skeleton path

Fig. 5 Discretization of corridors and path optimization using dynamic programming

$$U_k^g = \min_{g' \in \{1,2,\ldots,n_o(k+1)n_s+1\}} (u_k^{gg'} + U_{k+1}^{g'}) \ \ \forall g' \tag{14}$$

By setting $U_{n_p}^g = U_{n_p}^1 = 0$ at the goal when $k = n_p$ (and $g = 1$ since the goal point is the only possible candidate), Eq. (14) provides the Bellman equation, i.e., a recursive relation backward from the goal until we get $U_1^g = U_1^1$, which is

the minimum cost from the start to the goal. Our purpose is to find a path from the start to the goal that has the minimum cost U_1^1 and to find the combinations of g for all kth steps.

With the allowable speed defined in Eq. (8), the time of traveling to pass a point p can be expressed as

$$u(p) = \frac{\Delta p}{v_a(p)} \tag{15}$$

where Δp is the change of the position by infinitesimal amounts. We can apply Eq. (15) to define the cost $u_k^{gg'}$ as

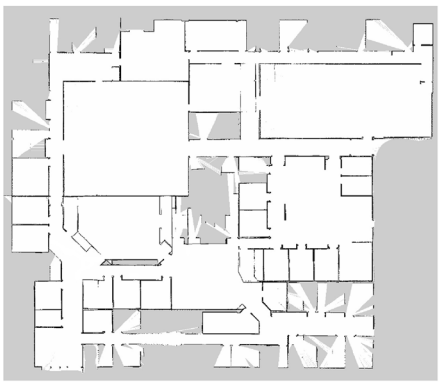
$$u_k^{gg'} := \frac{2\|s(k+1,g') - s(k,g)\|_2}{v_a(s(k+1,g')) + v_a(s(k,g))} \tag{16}$$

By using dynamic programming with Eqs. (14) and (16), we can obtain the set of g for all k for the minimum total cumulative cost and the set of off-skeleton points for the optimal path Q_p^*.

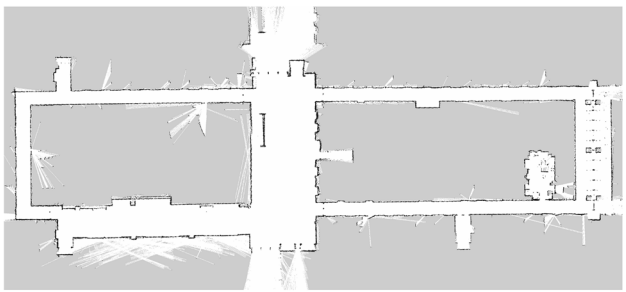
4 Simulation with real maps

In this section, we implement the proposed global path planning method with some real maps to show the optimality and computational efficiency of the resulting path. We compare the results with some existing path planning methods. Three real maps shown in Fig. 6 were used for our simulations: we downloaded the maps of Intel Research Lab [12] and Willow Garage [27] as shown in Figs. 6a, b, respectively, and created a map of E-7 building in our campus (University of Waterloo) as shown in Fig. 6c. The maps were chosen to encapsulate different characteristics of real environments that can demonstrate the benefits of our approach. For example, Intel Research Lab (Fig. 6a) has complex shapes of spaces, Willow Garage (Fig. 6b) has various sizes of rooms, and our campus building (Fig. 6c) has a mix of long and narrow corridors. We first visualize the HTM-EC of a real map and consequent path planning, see the optimal shape of the resulted path, and then look into the computation time of the proposed method.

(a) Intel Research Lab



(b) Willow Garage



(c) E-7 in University of Waterloo

Fig. 6 Maps used for the simulation of path planning

4.1 Visualization of HTM-EC and resulting paths with a real map

Figure 7 shows the visualization of HTM-EC and the path planning results with the map shown in Fig. 6a. In Fig. 7a, we showed only the skeleton of the map in green dots without the nodes and edges of the topology graph of HTM for better visibility. Two skeleton paths are shown in the figure with the start and goal positions. The path in the blue dashed line is the shortest skeleton path that does not consider the allowable speed of the robot, while the other path in the dashed orange line is the fastest skeleton path with the lowest cost based on the allowable speed. With the same velocity and acceleration setup that will be addressed in Sect. 5, the orange skeleton path planned by the HTM is about 4 m longer (68.8 m while the blue path is 64.9 m long), but its time of travel is expected to be 10.2 s shorter (45.2 s while the blue path is 35.0 s) provided that the robot is operated according to the allowable speed with respect to the corridor width. The average speed of the shortest path is 1.57 m/s, and that of the time-minimized path is 2.01 m/s.

In Fig. 7b, we visualized HTM-EC by showing the lines in cyan, $q(k)o_m(k)$, that connect the skeleton path $\mathcal{Q}_p$ and its boundary points. The skeleton path in orange color is the same as the fastest skeleton path from Fig. 7a. On the other hand, the final path colored in red is the one that is optimized from the discretized corridor of the HTM-EC. In Fig. 7c, we enlarged some right middle section of Fig. 7b.

4.2 Computation efficiency and path optimality

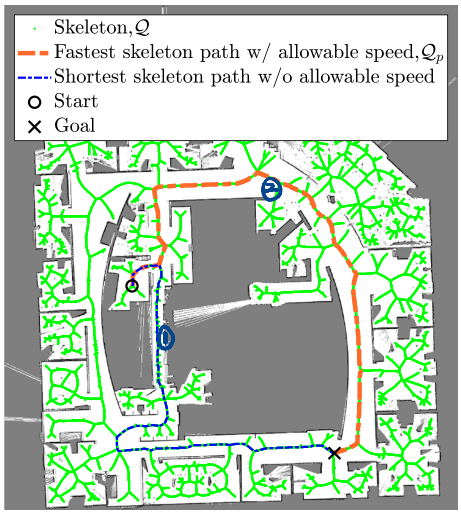
In this section, we perform the path planning simulation for the proposed method with real maps and compare it with existing methods. Two aspects of the comparison are considered: path optimality and computation time.

To compare the optimality of the path planned by HTM-EC, we implemented a grid-based global planning method equivalent to a widely used global planner in ROS (Robot Operating System), named *global planner* [4,20]. The algorithm utilizes a potential field created by grid-based Dijkstra expansion on an OGM until the expansion reaches a goal. Then, a path can be found by tracking the gradient of the potential back to the start position. We applied the proposed cost defined in Eq. (15) for each grid cell when the potential field is created so that the resulting path can be considered as the reference of the optimal path for the given cost. As the grid-based Dijkstra method is complete and known as the optimal solution, we can use it for the reference trajectory to compare with the proposed method for the optimality of resulting paths.

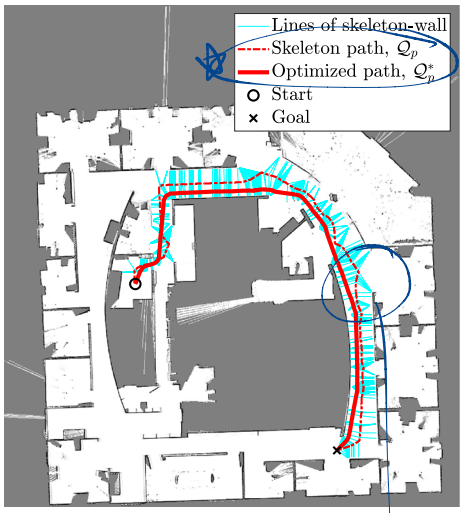
We also compare the proposed method with the RRT and some of its variants, particularly for the computation time, as those methods are known for fast algorithms [14,21]. We implemented the original version of RRT [19], RRT* [13], and RRT*J [34]. In RRT*J, the state-of-the-art RRT, skeleton information is used to create multiple potential functions for non-uniform sampling. This enables RRT samples to be scattered more likely around a feasible skeleton path. For the implemented RRT algorithms, we stopped the computation once a feasible path was found to get their minimized computation time. While there are plenty of other RRT variants [17,26], we selected those three algorithms because they can capture different characteristics: the original RRT shows the basic RRT characteristics (e.g., randomness), RRT* delivers a heuristic property, and RRT*J contains completeness.

For each map shown in Fig. 6, we simulated both methods for 200 cases with random start and goal positions, which were conducted by MATLAB R2020b with Intel i7-10750 H (2.6GHz) CPU and 16GB of memory. We compared the computation time and total cost from the simulations between HTM-EC planner and the other methods.

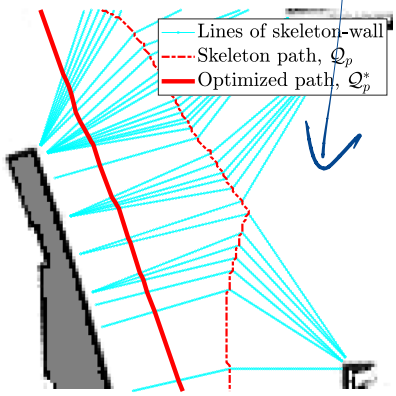
Figure 8 shows the comparison of the optimal paths between the HTM-EC and grid-based Dijkstra for the maps shown in Fig. 6b,c. Among the RRT paths, we only showed



(a) Skeleton path planning with HTM and the comparison of two paths with/without allowable speed

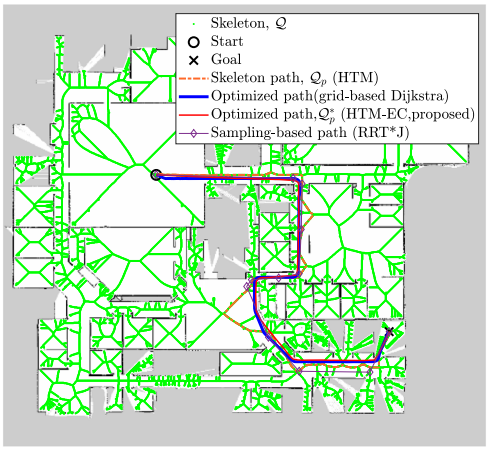


(b) Visualization of HTM-EC and the optimizd path from the skeleton path

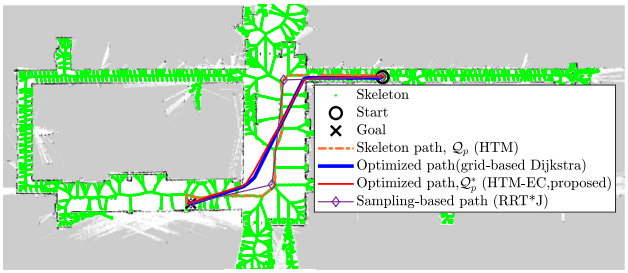


(c) Enlarged Fig. 7(b) for the middle-right parts of the path

Fig. 7 Path planning with HTM-EC using the map of Intel Research Lab



(a) map of Fig. 6(b)



(b) map of Fig. 6(c)

Fig. 8 Comparison of paths: skeleton path (HTM), optimized path by HTM-EC ($n_s = 50$), optimized path by the grid-based Dijkstra, and sampling-based path by RRT*J. Costs with the allowable speed were applied to all paths except the sampling-based path.

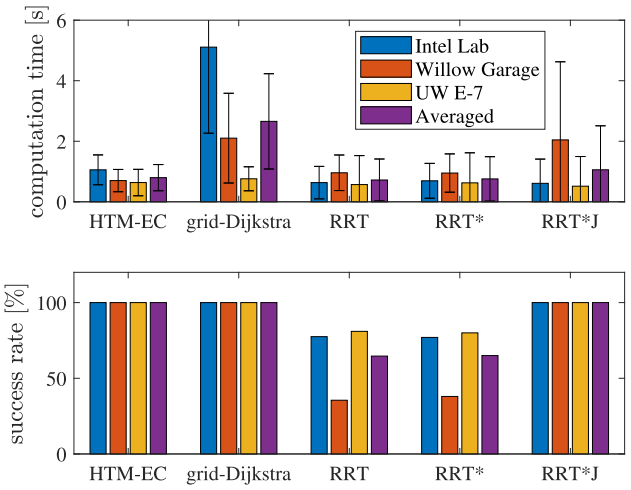


Fig. 9 Comparison of the computation time with three maps shown in Fig. 6. HTM-EC is the case when $n_s = 50$, which is the most right case of the bottom figure in Fig. 10

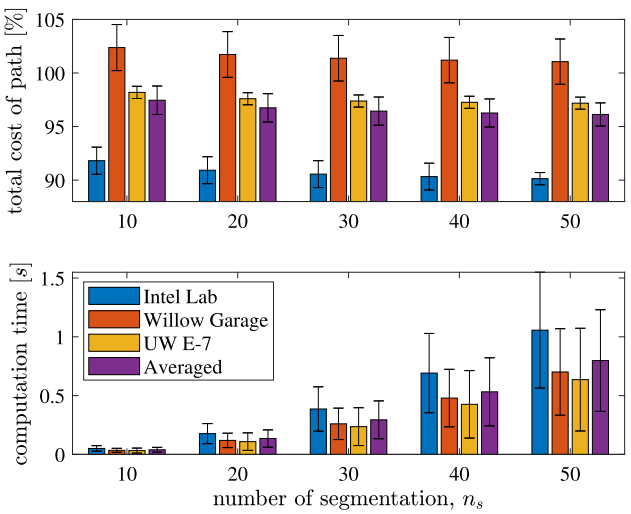


Fig. 10 Computation time (bottom) and total cost of paths (top) of HTM-EC planning with different number of corridor segmentation, n_s, for three maps shown in Fig. 6. The costs (lower is better) are represented in percentages over the cost of the grid-based Dijkstra path

the RRT*J path, which is likely stuck to a skeleton path, for better visibility. As RRT and RRT* often gave us totally different-shaped paths because of their randomness, showing their paths would not provide much meaning in the sense of optimality. The HTM-EC paths are similar in shape to the grid-based Dijkstra paths, qualitatively implying their optimality.

Table 1 and Fig. 9 summarize the comparison of computation time and the total costs. The results of HTM-EC shown Fig. 9 for the comparison with other methods are for $n_s = 50$ for each line of $\overline{q(k)o_m(k)}$.

We computed the total costs of different methods in percentages over those of the grid-based Dijkstra path, as the absolute costs can largely depend on the random start and goal positions. Based on the total costs, the results show, in general, that HTM-EC paths have lower (better) costs over the grid-based Dijkstra paths: 9.86% and 2.82% lower in Intel Lab and UW E-7 maps, 1.06% higher in Willow Garage. Noting that the skeleton planning step of HTM-EC is exactly the same as HTM, our previous work, the results also show substantial improvements in optimality over the previous HTM method: 22.54%, 19.87%, and 17.44% in each of the three maps, respectively.

Besides the optimal path shape, the computational load is another important criterion for a path planner. Overall, the averaged computation time of HTM-EC with (0.7978 s, $n_s = 50$ case) is much shorter than the grid-based Dijkstra method (2.6574 s) and shorter than RRT*J (1.0572 s). RRT (0.7206 s) and RRT* (0.7565 s) are slightly faster, but they often fail to find a path as shown in their success rate in the table. The mean and standard deviation values of computation time are shown in the table for each map. With the fewer number of sampling, $n_s = 10$, the computation of HTM-EC becomes even much faster about 20 times, resulting in 0.0389 s on

average, as shown in the table for each map. Figure 10 shows the computation time of HTM-EC and the total cost as its tradeoff with different sampling numbers n_s from 10 to 50, two of which are shown in Table 1.

One interesting point is that the reduction in the computation with HTM-EC is significantly larger in the first two maps (the maps for Intel Research Lab and Willow Garage) than that in the third map (E-7 map), which comes from the complexity of a map. The third map has simpler branches of corridors compared to the other two maps, making the Dijkstra expansion easier (and consequently faster) than the other two maps. In the first two maps, the first is noisier (thus, more complex) than the second map (seen as many small spots in the upper right chamber of the first map), causing longer computation time for the grid-based Dijkstra expansion.

Another point to mention is the variances of the computation time for three maps that comes with different complexities. The computation time of the proposed method shows smaller variance (0.64-−1.06 s) than that of the grid-based Dijkstra method (0.76-−5.12 s). It is notable that the standard deviation of the overall computation time for each map also has smaller variance with HTM-EC (0.37-−0.49 s) than the grid-based Dijkstra method (0.40-−2.84 s).

Similar points can be noted in the RRT methods. Although their overall planning time has less difference than the grid-based Dijkstra method, their high standard deviations, which can be even larger than the mean planning time, imply the planning time significantly varies. This is because of the randomness of the sampling-based tree expansion, often reaching to the maximum iteration when failing to find a path. All of three RRT methods showed the worst performance in computation time in the map of Willow Garage (Fig. 6b). The relatively small sizes of doors of many chambers in the map can cause the planning failure while there are large spaces, because it more likely results in the collision of a expanding-line of RRT tree that connects a randomly deployed sample to the existing RRT tree. The large chambers have more probabilities for new samples, but these samples are less likely seen by other samples out of the chambers. RRT*J showed successes in all simulations thanks to the informed sampling by the skeleton path, but the large spaces degraded its computation efficiency because more computation was required to get its multiple potential function with larger clearances.

We need to look at more details regarding computation time and variances with the compositions of the grid-based Dijkstra method and the RRT methods. Considering that the grid-based Dijkstra method takes two steps (i.e., Dijkstra expansion in a grid map and gradient tracking), its computation time can be viewed for each step. Note that each cell in the free space (white areas of an OGM) can be regarded as a node for Dijkstra expansion in a grid map; thus, the number of nodes and edges are large, resulting in heavy computation. For example, the number of nodes for the first map (Intel Research Lab) is 147,278 in the worst case when the Dijkstra expansion fully covers the free space of the map (in comparison, the topology graph of HTM-EC for the same map has 1,372 nodes, 1,522 edges, and 18,054 skeleton points). In fact, the step for Dijkstra expansion takes most of the computation time for the grid-based Dijkstra method, about 94.37% of the total time (97.16, 94.07, and 91.88% for each map, respectively). The RRT methods also required a significant portion of their planning time for the tree-graph expansion, which is more than 99.9% of the total computation for all three variants. The expansion of the graph is required when replanning is requested. When a map becomes larger or more complex, it will likely need more time for Dijkstra expansion and RRT tree expansion. Even on the same map, the time elapsed can vary in a wide range depending on the start and goal locations. For example, the Dijkstra expansion will quickly reach a goal if the goal is close to the start, while it will take more time when the goal is set far from the start location.

The computation time with HTM-EC can also be viewed in two steps: skeleton path planning and its optimization. Compared to the grid-based Dijkstra method and the RRTs, the computation time for graph search on the topology graph of HTM-EC is almost immediate (about 3.6 ms in average, leading to 0.47% for $n_s = 50$ and 9.74% for $n_s = 10$) due to the already constructed searching space with fewer numbers of nodes and edges (e.g., 1,372 and 1,522, respectively, as mentioned above). Instead, most of the computation time with HTM-EC is taken by the step for optimizing a skeleton path (about 99.53% for $n_s = 50$ and 90.26% for $n_s = 10$).

It is worth noting that an on-skeleton path, $\mathcal{Q}_p$, is a feasible path and available at a very early time, although further optimization is required to improve its quality at the later time of computation. This is always the case regardless of different sampling of n_s on each line of $\overline{q(k)o_m(k)}$. n_s, and the computation time for the optimization can even be significantly reduced with fewer segmentations as shown in Fig. 10 and Table 1. Our optimization strategy utilizes dynamic programming, so the complexity of the optimization algorithm is the same as that of dynamic programming which is $O(n^2)$. However, the size of n, in our case n_s, is significantly reduced, and it can be observed in Figs. 9 and 10 that the actual computation is fast. All of these observations not only show that the computation time can be reduced with HTM-EC but also imply that it will be even more advantageous when a map becomes larger and more complex.

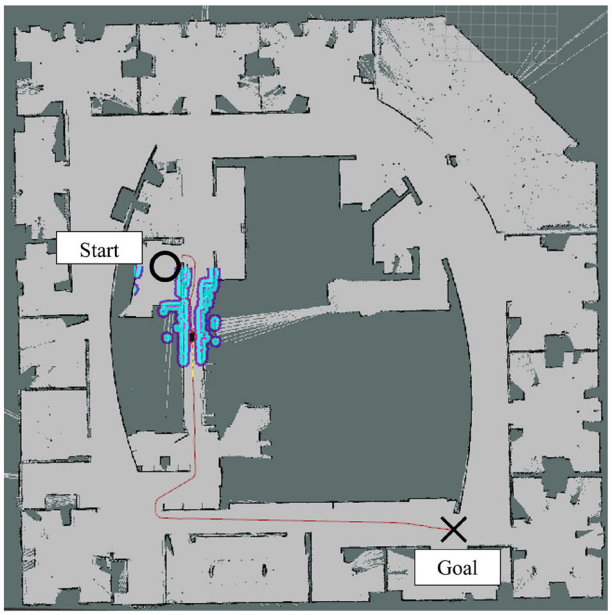
5 Robot implementations: verification of the allowable speed

While Sect. 4 shows us that the proposed path planning method is implementable with real maps, we need further implementations on a real robot to see if the proposed cost

Table 1 Comparison of the simulation results for the computation time, total cost, and success rate with the maps shown in Fig. 6

Method	Step	Intel Lab			Willow Garage			UW E-7		
		Time (s)	Cost (%)	Success (%)	Time (s)	Cost (%)	Success (%)	Time (s)	Cost (%)	Success (%)
	Skeleton planning	0.0040 (0.0021)	112.68 (9.77)	100.0	0.0032 (0.0015)	120.93 (29.32)	100.0	0.0036 (0.0026)	114.62 (0.3248)	100.0
HTM-EC	Optimization	1.0525 (0.4913)	–	–	0.6978 (0.3672)	–	–	0.6322 (0.4346)	–	–
($n_s = 50$)	Total	1.0565 (0.4926)	**90.14 (5.66)**	100.0	0.7010 (0.3682)	101.06 (21.14)	100.0	0.6358 (0.4368)	**97.18 (5.62)**	100.0
	Skeleton planning	0.0040 (0.0021)	112.68 (9.77)	100.0	0.0032 (0.0015)	120.93 (29.32)	100.0	0.0036 (0.0026)	114.62 (0.3248)	100.0
HTM-EC	Optimization	0.0464 (0.0223)	–	–	0.0312 (0.0161)	–	–	0.0283 (0.0192)	–	–
($n_s = 10$)	Total	**0.0504 (0.0236)**	91.81 (12.66)	100.0	**0.0343 (0.0170)**	102.37 (21.50)	100.0	**0.0319 (0.0214)**	98.19 (5.70)	100.0
	Dijkstra expansion	4.9643 (2.8399)	–	–	1.9781 (1.4801)	–	–	0.6985 (0.3946)	–	–
grid-based	Planning (gradient)	0.1451 (0.0192)	–	100.0	0.1246 (0.0156)	–	100.0	0.0617 (0.0080)	–	100.0
Dijkstra	Total	5.1094 (2.8435)	100.0*	100.0	2.1027 (1.4822)	**100.0***	100.0	0.7602 (0.3967)	100.0*	100.0
	Tree-graph expansion	0.6337 (0.5361)	–	–	0.9585 (0.5878)	–	–	0.5689 (0.9572)	–	–
RRT	Planning	0.0002 (0.0005)	–	–	0.0002 (0.0009)	–	–	0.0001 (0.0002)	–	–
	Total	0.6340 (0.5359)	123.16 (37.12)	77.5	0.9587 (0.5875)	135.5 (50.48)	35.5	0.5691 (0.9571)	123.54 (35.37)	81.0
	Tree-graph expansion	0.6930 (0.5759)	–	–	0.9498 (0.6327)	–	–	0.6259 (0.9931)	–	–
RRT*	Planning	0.0006 (0.0044)	–	–	0.0001 (0.0003)	–	–	0.0001 (0.0002)	–	–
	Total	0.6936 (0.5758)	108.71 (24.84)	77.0	0.9499 (0.6326)	121.69 (37.56)	38.0	0.6260 (0.9930)	107.24 (18.82)	80.0
	Tree-graph expansion	0.6103 (0.8002)	–	–	2.0449 (2.5801)	–	–	0.5158 (0.9798)	–	–
RRT*J	Planning	0.0003 (0.0005)	–	–	0.0002 (0.0003)	–	–	0.0002 (0.0002)	–	–
	Total	0.6107 (0.8003)	95.88 (6.01)	100.0	2.0451 (2.5801)	101.28 (8.81)	100.0	0.5159 (0.9798)	100.90 (8.39)	100.0

Bold values emphasize the best performance
Standard deviation is written in parentheses if available
*The grid-based paths were used for the reference costs



(a) Simulations with Intel Lab: the shortest path (the dark red line)

(b) Simulations with Intel Lab: longer, but faster path by the proposed cost (the dark red line)

Fig. 11 Comparison of the shortest paths and time minimized paths with ROS simulations with Intel Lab

can lead to a faster path while it is longer in length. Therefore, we have performed ROS simulations and experiments to verify our proposed method for time-minimization with a real mobile robot. Through the robot simulations and experiments, we compare the time of traveling between the shortest path, planned by the *global planner* in ROS, and the time-minimized path, planned by HTM-EC, with the allowable speed and the proposed cost. The ROS planner does not use the proposed cost, hence returns the shortest path by default. A ROS local planner, *TEB (Timed-Elastic-Band) local planner* [28], was used for both global planners during the implementations in this section.

Table 2 ROS simulations: total length of the path, average speed and total time for the cases (Intel Lab)

	Fastest path (proposed)	Shortest path
Path length (m)	64.3486 (std 0.1563)	50.0488 (std 0.2538)
Time of traveling (s)	30.9480 (std 0.2891)	34.3320 (std 0.5051)
Avg. speed (m/s)	1.9995 (std 0.0069)	1.4570 (std 0.0162)

An omnidirectional mobile robot, Summit XLS manufactured by Robotnik Inc., was used for simulations and experiments with the same parameters and conditions (see Fig. 13a in the later section).

For the *TEB local planner*, we set it with a 8 m × 8 m in the local map with small sideways velocity, i.e., $v_x > v_y$, where positive x is forward, and positive y is sideways to the left of the mobile robot. The maximum velocity in the x direction was set as 3 m/s and y for 0.5 m/s, and the acceleration was set 1.5 $\mathrm{m/s^2}$. The allowable speed was applied to the *TEB local planner* for both global plans. While the robot was moving, its states were obtained by the *AMCL (Adaptive Monte Carlo Localization)* algorithm in ROS with a *VLP-16* LiDAR. The estimated robot's location and clearance from the obstacles were used to adjust the robot's actual navigation speed under the allowable speed. With this setup, we performed simulations with two different environments, Intel Lab (Fig. 6a) and UW E-7 (Fig. 6c). We also performed a real robot experiments with the UW E-7 map.

5.1 Scenario 1: Robot simulations with Intel Lab Map

Figure 11 describes the simulation scenario by showing the start and goal positions and the different paths between the shortest (Fig. 11a) and time-minimized with the allowable speed (Fig. 11b), which gives us the different corridor selections. The shortest path takes the downward corridor, while the time-minimized path selects the upward corridor for larger clearances and faster speed in operation. The wider corridors enable the robot to move faster with higher speed commands shown in Fig. 12, and the time of traveling can be reduced as a result. We performed ten simulations to check the consistency of the results and summarized the results in Table 2. The time-minimized path is longer, about 14.3 m in length, but 3.4 s faster than the shortest path. Compared to the skeleton path shown in Fig. 7a, the path length was reduced to 64.3 from 68.9 m, and the time of traveling was also reduced to 30.9 from 35.0 s (Fig. 13).

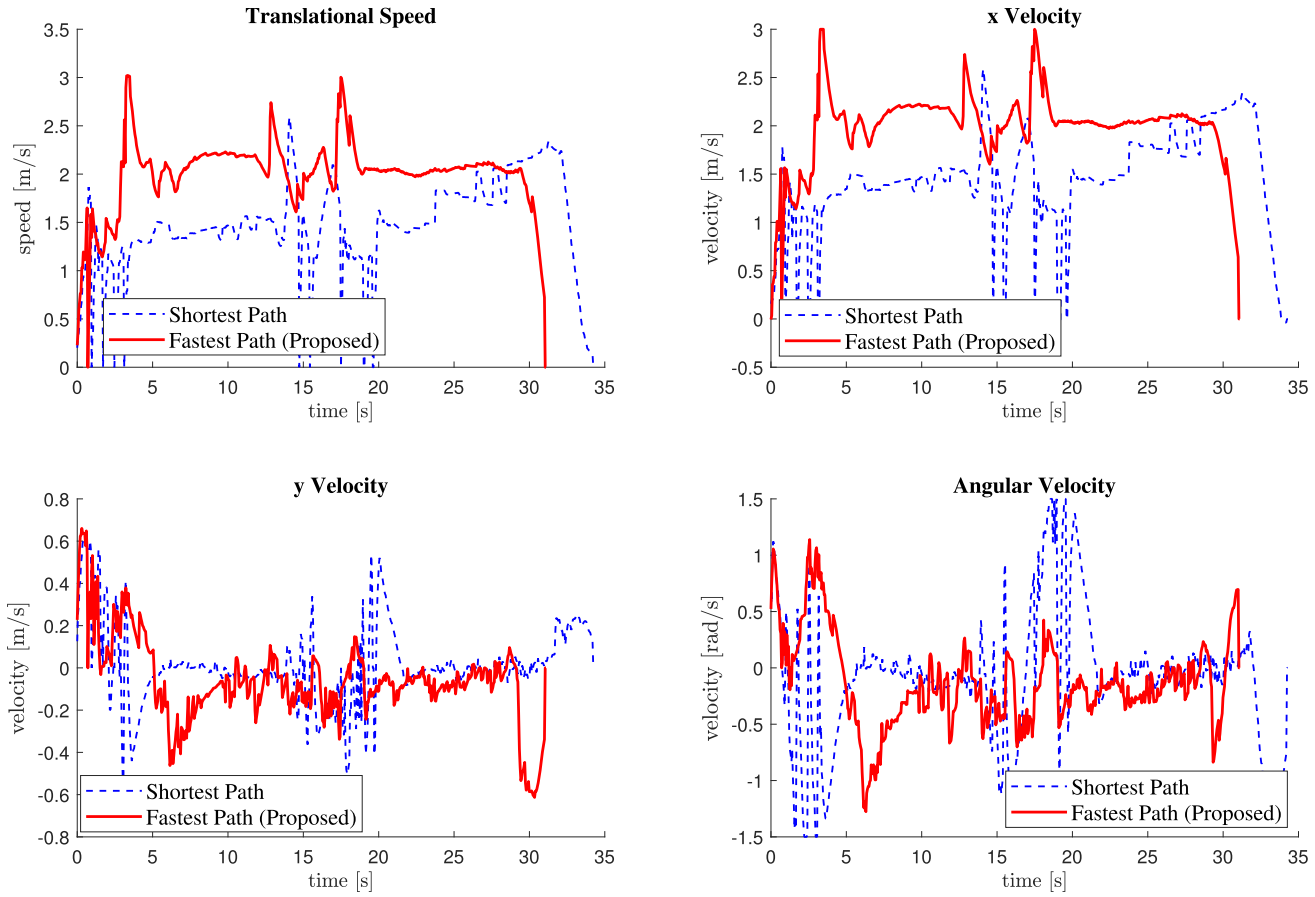


Fig. 12 Comparison of the actual speed along the paths in the simulations (Intel Lab)

5.2 Scenario 2: Robot simulations and experiments with UW E-7 map

Figure 13a shows the mobile robot, and Fig. 13b shows the robot running in a corridor to show the environment of the experiments. We used the map of the E-7 building on our campus (Fig. 6c) for the ROS simulations and performed the experiments in the same place with the same local planner and its parameters.

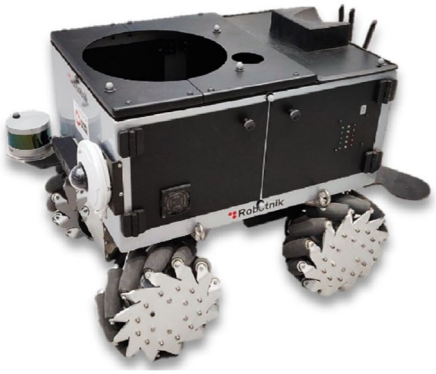
Figure 14a describes the simulation environments of UW E-7 map in Gazebo, a ROS simulation tool. We set the start position in the middle of the left corridor and the goal at the slightly upper location in the right lobby. It is easy to expect, as shown in Fig. 14a, that the upper corridor will provide the shortest path, whereas the lower corridor can provide the fastest path, even with the longer distance, thanks to its large widths that enable higher speed operation. We set up the same start and goal locations for the experiments, and both the simulations and experiments showed us the fastest path via the lower corridor by our HTM-EC planner and the shortest path via the upper corridor by the ROS default global planner. We performed 20 simulations and 12 experiments with UW E-7 map, and showed the path comparison for the experiments in Fig. 14b. In the figure, we used the dashed lines for the planned paths and solid lines for the actual paths that the robot took during the experiments.

We summarized the results from the simulations and the experiments with the UW E-7 map in Tables 3 and 4, respectively. The lower paths, generated by the proposed method, are about 6–7 s faster, although their lengths are about 5–7 m longer due to the high-speed operation captured in Fig. 15. The videos of the experiments can be accessed from the link in the footnote.[1]

6 Discussion

In this paper, we introduced a path planning method that utilizes *allowable speed* to explicitly consider safety in the structure of HTM-EC. Through simulations with real maps, it has been shown that the usage of the sparse structure of HTM-EC allowed lower computation time than the conventional method of potential fields with grid-based Dijkstra expansion, while the completeness of the method is achieved.

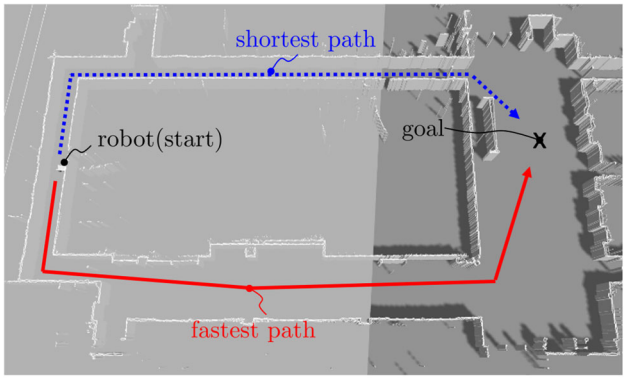
[1] https://youtu.be/oDTTdYcj_qA.



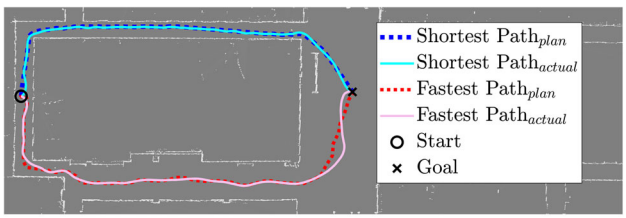
(a) Mobile robot

(b) Mobile robot navigating in the experiment

Fig. 13 Environment for the experiments



(a) Simulation environments



(b) Comparison of the paths in the experiments

Fig. 14 Simulation environments and path comparison in the experiments

Table 3 ROS simulations: total length of the path, average speed and total time for the cases (UW E-7)

	Fastest path (proposed)	Shortest path
Path length (m)	55.0191 (std 0.1074)	50.8499 (std 0.0324)
Time of traveling (s)	28.8082 (std 0.4703)	35.6360 (std 0.2913)
Avg. speed (m/s)	1.9692 (std 0.0089)	1.4424 (std 0.0093)

Table 4 Experiments: total length of the path, average speed and total time for the cases shown in Fig. 14b (UW E-7)

	Fastest path (proposed)	Shortest path
Path length (m)	58.8000 (std 0.0809)	51.5589 (std 0.0919)
Time of traveling (s)	29.5337 (std 1.8513)	35.3321 (std 1.1108)
Avg. speed (m/s)	2.0448 (std 0.0937)	1.4586 (std 0.0380)

Specifically, we expect the advantage of less computation time by the proposed method will be even more highlighted when a map becomes large and complex due to the low variances of the computation. The planned global path by the proposed method is refined from an on-skeleton path to be smooth and optimal.

As we have verified through the implementations to a real robot, the use of allowable speed for the global path planning provided the robot with a faster path than the shortest path with the preference to the spacious areas for the path planning. The robot had no problem increasing its speed in the wider corridors that eventually allowed the longer path to be faster.

6.1 Issues

There are several implementation issues that we would like to discuss further. Increasing the number of segmentation of the line between a skeleton point and its closest obstacles will make the resulting path closer to the optimal with the finer refinement. However, we recommend keeping a certain value for the number of segmentation, as the quality of the path would be good enough for a global path. (In practice, the local path planners will modify the global path in the proximity to take care of the changes of environments.)

It is also noted that, for some of the skeleton points, we did not obtain the closest points on the obstacles (i.e., walls). We used the image-thinning method to get the skeleton from a map which resulted in the pixel-based integer values of (x, y) for a skeleton point. Thus, the skeleton point may be slightly biased to one of the boundary points, and the boundary points, at least two, may give slightly different distances from the related skeleton point, resulting in failure to detect the obstacle points with the exact same distance from some

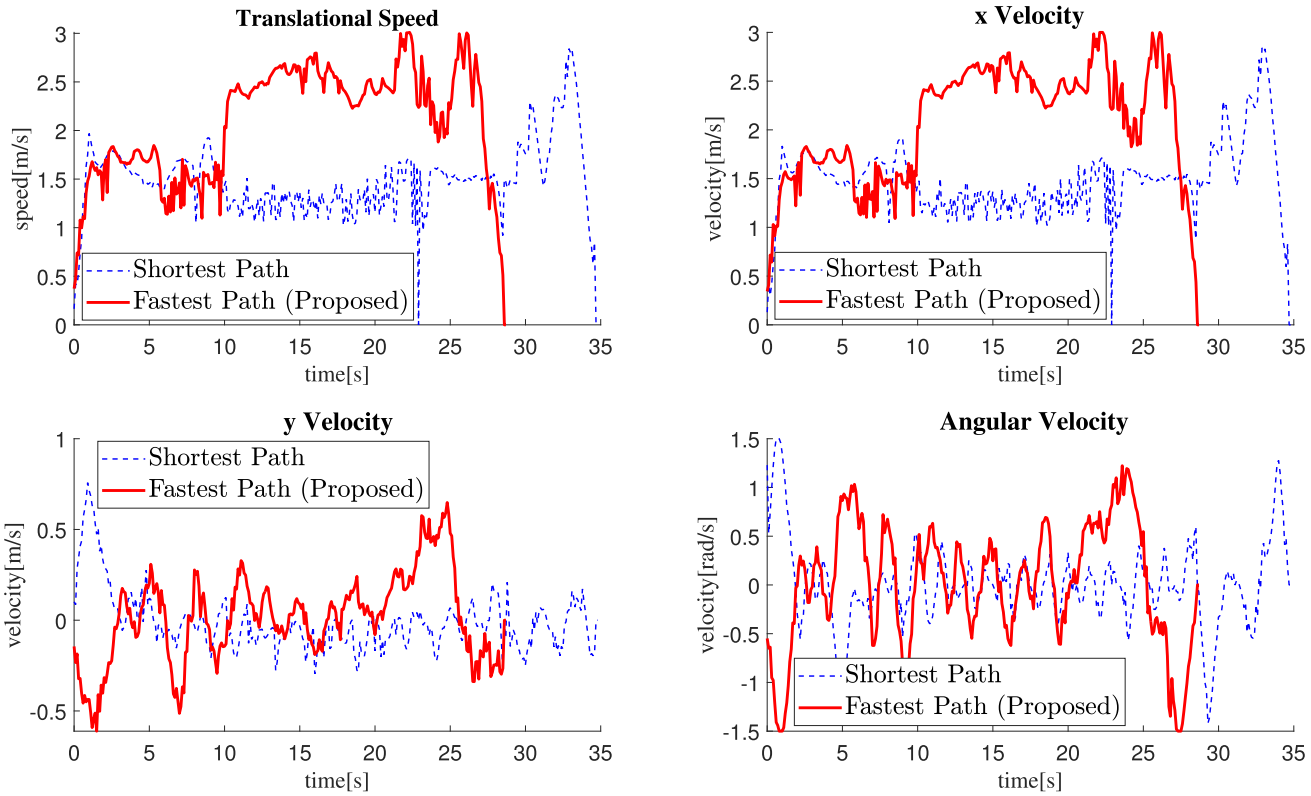


Fig. 15 Comparison of the actual speed along the paths in the experiments (UW E-7)

of the skeleton points. This issue can be resolved by applying other skeletonization methods, such as the Voronoi diagram, which provides us with the exact values of (x, y) for the skeleton points.

更精确

6.2 Future works

As topological planning takes the abstract cost accumulation of corridors, one interesting future work can be related to adjusting corridor costs in real-world applications. In warehouses, for example, where the environments are mostly static and well-constructed but still have some dynamic objects. In such cases, even large regions can be slower corridors if they are often crowded with moving obstacles, degrading the actual navigation time. We expect such flexibility of adjustment of the corridor costs would help the time performance of navigation robots in real-world situations.

Another future work will be on the extended use of the proposed method for the local planning aspect. As the topology graph of HTM-EC contains the intersection of corridors, we can use HTM-EC for other applications of mobile robot navigation, such as the motion prediction of other moving agents in the surrounding based on the intersection information incorporated in the graph. Considering most dynamic agents like humans often move toward the intersections, it may help predict their motion by presuming their short-term goals on nodes of the topology graph of the HTM-EC and by inferring their goals.

Acknowledgements This work was supported by Natural Sciences and Engineering Research Council (NSERC) of Canada under the Strategic Partnership Grant for Projects (STPGP-506987-2017). The authors would like to thank James Kattukudiyil for his support for the code implementation and Sooyong Kim for his support during the experiment.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Appendix A

Algorithm 1: Construction of Hierarchical Topology Map from skeleton

Input: unexplored skeleton set $\mathcal{P}$, branching points $\mathcal{P}_{br}$, distance field of a map D
Output: topology node set $\mathcal{N}$, topology edge set $\mathcal{E}$, ordered skeleton set $\mathcal{Q}$

/* Initialization */
$i, j, I_i, I_j \leftarrow 1$ // $I_{ij} = \{I_i, I_j\}$
SearchPt$\leftarrow$argmax$_{p\in\mathcal{P}}(D(p))$
SearchQueue.add, $\mathcal{N}$.add$\leftarrow$SearchPt
$\mathcal{P}$.remove $\leftarrow$SearchPt

Algorithm 2: Construction of Hierarchical Topology Map from skeleton (continued)

```
   /* Explororation of unexplored skeleton   */
5  while SearchQueue is not empty do
6      Neighbor ←8NeighborWindow + SearchPt;
7      SearchQueue.remove ←SearchPt;
       /* add unexplored points to queue      */
8      for each Neighbor do
9          if Neighbor is found from P then
10             if Neighbor is found from P_br then
11                 NeighborBranch.add ←Neighbor
12             else
13                 SearchQueue.add←Neighbor
14     SearchQueue.add←NeighborBranch;
15     Q.add←SearchPt;
16     P.remove ←SearchPt;
       /* case1: newly found branching point  */
17     if SearchPt is found from P_br then
18         N.add ←SearchPt j←length(N), I_j ←length(Q);
19         E.add ←(i, j, I_i, I_j);
20         Q.add←SearchPt;
21         i←j, I_i ←length(Q);
       /* case2&3: merging or terminal points */
22     if Neighbor is not found from P then
           ;              // next SearchPt will jump
23         SearchPtJump←true;
24         if Neighbor∈N then
25             j←findindex(N = Neighbor), I_j ←length(Q);
26             E.add ←(i, j, I_i, I_j) ; // merging node case
27         else
28             N.add ←SearchPt ;
29             j←length(N), I_j ← length(Q);
30             E.add ←(i, j, I_i, I_j) ; // terminal node case
31     SearchPt←SearchQueue.last ;
       /* find adjacent nodes if SearchPt jumps
          */
32     if SearchPtJump is true then
33         Neighbor = 8NeighborWindow + SearchPt;
34         SearchQueue.remove ←SearchPt;
35         for each Neighbor do
36             if Neighbor∈N then
37                 i ←findindex(N = Neighbor);
38                 Q.add ←Neighbor;
39                 I_i ←length(Q);
40         CandidateJump = false
   /* Compute allowable speed and edge cost  */
41 for each Q do
42     compute v_a(Q), u(Q);
43 for each E do
44     compute C=sum(u) ;          // compute edge cost
45     E.concatenate ← C
46 return N, E, Q
```

References

1. Bellman R (1966) Dynamic programming. Science 153(3731):34–37
2. Bhattacharya P, Gavrilova ML (2007) Voronoi diagram in optimal path planning. In: 4th IEEE international symposium on Voronoi diagrams in science and engineering (ISVD 2007), pp 38–47
3. Bhattacharya P, Gavrilova ML (2008) Roadmap-based path planning—using the Voronoi diagram for a clearance-based shortest path. IEEE Robot Autom Mag 15(2):58–66. https://doi.org/10.1109/MRA.2008.921540
4. Brock O, Khatib O (1999) High-speed navigation using the global dynamic window approach. In: Proceedings 1999 IEEE international conference on robotics and automation (Cat. No. 99CH36288C), vol 1. IEEE, pp 341–346
5. Choset H, Burdick J (2000) Sensor-based exploration: the hierarchical generalized Voronoi graph. Int J Robot Res 19(2):96–125
6. Dijkstra EW (1959) A note on two problems in connexion with graphs. Numerische Mathematik 1(1):269–271
7. Dong Y, Camci E, Kayacan E (2018) Faster rrt-based nonholonomic path planning in 2d building environments using skeleton-constrained path biasing. J Intell Robot Syst 89(3):387–401
8. Geraerts R (2010) Planning short paths with clearance using explicit corridors. In: 2010 IEEE international conference on robotics and automation. IEEE, pp 1997–2004
9. Gómez JV, Mavridis N, Garrido S (2014) Fast marching solution for the social path planning problem. In: IEEE international conference on robotics and automation (ICRA), pp 1871–1876
10. Han Jw, Jeon S, Kwon HJ (2019) A new global path planning strategy for mobile robots using hierarchical topology map and safety-aware navigation speed. In: 2019 IEEE/ASME international conference on advanced intelligent mechatronics (AIM). IEEE, pp 1586–1591
11. Hart PE, Nilsson NJ, Raphael B (1968) A formal basis for the heuristic determination of minimum cost paths. IEEE Trans Syst Sci Cybern 4(2):100–107
12. Holz D, Behnke S (2010) Intel Research Lab (Seattle). http://www.ais.uni-bonn.de/~holz/spmicp/files/intel_map.gif @ONLINE. Accessed 15 Jan 2019
13. Karaman S, Frazzoli E (2011) Sampling-based algorithms for optimal motion planning. Int J Robot Res 30(7):846–894
14. Karur K, Sharma N, Dharmatti C, Siegel JE (2021) A survey of path planning algorithms for mobile robots. Vehicles 3(3):448–468
15. Kavraki LE, Kolountzakis MN, Latombe JC (1996) Analysis of probabilistic roadmaps for path planning. In: IEEE international conference on robotics and automation (ICRA), vol 4, pp 3020–3025
16. Khatib O (1985) Real-time obstacle avoidance for manipulators and mobile robots. In: Proceedings of the 1985 IEEE international conference on robotics and automation, vol 2. IEEE, pp 500–505
17. Kiani F, Seyyedabbasi A, Aliyev R, Gulle MU, Basyildiz H, Shah MA (2021) Adapted-rrt: novel hybrid method to solve three-dimensional path planning problem using sampling and metaheuristic-based algorithms. Neural Comput Appl 33(22):15569–15599
18. Kunz T, Stilman M (2015) Kinodynamic rrts with fixed time step and best-input extension are not probabilistically complete. In: Algorithmic foundations of robotics XI. Springer, pp 233–244
19. LaValle SM, Kuffner JJ, Donald B et al (2001) Rapidly-exploring random trees: progress and prospects. Algorithm Comput Robot New Direct 5:293–308
20. Marder-Eppstein E, Lu DV (2013) ROS global planner. https://github.com/ros-planning/navigation/blob/noetic-devel/global_planner/src/dijkstra.cpp @ONLINE. Accessed 30 July 2020

21. Moll M, Sucan IA, Kavraki LE (2015) Benchmarking motion planning algorithms: an extensible infrastructure for analysis and visualization. IEEE Robot Autom Mag 22(3):96–102
22. Ogniewicz R, Ilg M (1992) Voronoi skeletons: theory and applications. In: IEEE computer society conference on computer vision and pattern recognition, pp 63–69
23. Orozco-Rosas U, Montiel O, Sepúlveda R (2019) Mobile robot path planning using membrane evolutionary artificial potential field. Appl Soft Comput 77:236–251
24. Orozco-Rosas U, Picos K, Montiel O (2019) Hybrid path planning algorithm based on membrane pseudo-bacterial potential field for autonomous mobile robots. IEEE Access 7:156787–156803
25. Paden B, Čáp M, Yong SZ, Yershov D, Frazzoli E (2016) A survey of motion planning and control techniques for self-driving urban vehicles. IEEE Trans Intell Veh 1(1):33–55
26. Pradhan S, Mandava RK, Vundavilli PR (2021) Development of path planning algorithm for biped robot using combined multi-point rrt and visibility graph. Int J Inf Technol 13(4):1513–1519
27. Robotnik Inc (2017) Willow Garage Map. https://github.com/RobotnikAutomation/summit_xl_common/blob/kinetic-devel/summit_xl_localization/maps/willow_garage/willow_garage.pgm@ONLINE. Accessed 30 July 2020
28. Rösmann C, Feiten W, Wösch T, Hoffmann F, Bertram T (2012) Trajectory modification considering dynamic constraints of autonomous robots. In: ROBOTIK 2012; 7th German conference on robotics. VDE, pp 1–6
29. Sethian JA et al (2003) Level set methods and fast marching methods. J Comput Inf Technol 11(1):1–2
30. Song R, Liu Y, Bucknall R (2017) A multi-layered fast marching method for unmanned surface vehicle path planning in a time-variant maritime environment. Ocean Eng 129:301–317
31. Thrun S (1998) Learning metric-topological maps for indoor mobile robot navigation. Artif Intell 99(1):21–71
32. Tsardoulias EG, Iliakopoulou A, Kargakos A, Petrou L (2016) A review of global path planning methods for occupancy grid maps regardless of obstacle density. J Intell Robot Syst 84(1):829–858
33. van Toll W, Cook AF IV, van Kreveld MJ, Geraerts R (2017) The explicit corridor map: a medial axis-based navigation mesh for multi-layered environments. arXiv preprint arXiv:1701.05141
34. Wang J, Meng MQH (2020) Optimal path planning using generalized Voronoi graph and multiple potential functions. IEEE Trans Ind Electron 67(12):10621–10630
35. Wang Q, Langerwisch M, Wagner B (2013) Wide range global path planning for a large number of networked mobile robots based on generalized Voronoi diagrams. IFAC Proc Vol 46(29):107–112
36. Zhang T, Suen CY (1984) A fast parallel algorithm for thinning digital patterns. Commun ACM 27(3):236–239

Jeong-woo Han Jeong-woo Han s a Postdoctoral Fellow in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. His research interests lie in motion planning and control in robotics, covering mobile robots and serial robots. He received his B.S. and M.S. in Mechanical Engineering at Yonsei University in 2010 and 2012 and his Ph.D. in Mechanical and Mechatronics Engineering at the University of Waterloo in 2022. He has five years of industrial experience as a robotics researcher at Hyundai Heavy Industries in South Korea until 2017, which comprises the design and motion control of serial robots.

Soo Jeon Soo Jeon is a Professor in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. Professor Jeon's expertise lies in the broad areas of mechatronics, dynamic systems, instrumentation and control. His research projects include Sensing-Rich Drive Trains, Multimodal Sensory Feedback for Dexterous Manipulation, Non-Linear Dynamics of Controlled Mechanical Systems, Autonomous and Self-Sustaining Systems. Professor Jeon is the recipient of the Rudolf Kalman Best Paper Award from the American Society of Mechanical Engineers (ASME) for the year 2010 and the Discovery Accelerator Supplement Award from the NSERC for the year 2015. He is a member of ASME, IEEE and CSME. He serves as an associate editor for ASME Journal of Dynamic Systems, Measurement and Control, and IEEE Transactions on Automation Science and Engineering.

Hyock Ju Kwon HJ Kwon is an Associate Professor in the Department of Mechanical and Mechatronics Engineering at the University of Waterloo. His research interests include AI and machine learning, AI for manufacturing, ultrasound nondestructive testing (NDT),on-board AI for NDT, biomedical therapeutic and diagnostic ultrasounds and Fatigue and Fracture. His research group is actively developing various cutting-edge technologies that can make innovation and impact to manufacturing and biomedical sectors.